



Obligatorio de bioinformática 1

Ingeniería en Biotecnología y Licenciatura en Biotecnología

Nazareno Iván Cabrera (28746)

Ángela Tesitore (255962)

Índice de Contenidos:

- **Ejercicio 1:**
 - 1.1. Parte A.....1
 - 1.2. Parte B.....3
 - 1.3. Parte C.....3
- **Ejercicio 2:**
 - 2.1. Parte A.....8
 - 2.2. Parte B.....9
- **Ejercicio 3:**
 - 3.1. Parte D.....11
 - 3.2. Parte E.....11
 - 3.3. Parte F.....12

Ejercicio 1:

Se comenzó creando un directorio de trabajo con el comando *mkdir* con el fin de tener un espacio de trabajo organizado y se ingresó al mismo.

```
mkdir Obligatorio_2024
cd Obligatorio_2024
```

Posteriormente, se buscó en NCBI las bacterias asignadas con los accesions GCF_001617565.1 y GCF_001900355.1 para bacteria 1 y 2, respectivamente. Ambas bacterias corresponden a la especie *E.coli* siendo la primera la cepa *Ecol_732* y la segunda C11. Se ingresa a FTP para poder copiar la dirección de enlace de los archivos con los genomas completos de ambas cepas (archivos con .fna) para la parte a y los archivos con las regiones codificantes para la parte b y c (archivos con .ffn)). Tras copiar la dirección de enlace, se utilizó el comando *wget* en Bash para crear los archivos correspondientes a cada tipo de secuencia biológica de cada cepa, seguido del comando *gunzip* para descomprimirlos. Asimismo, se renombraron los archivos para un mejor manejo de los mismos copiando su contenido

Ejemplo con archivo de secuencia nucleotídica del genoma completo de cepa *Ecol_732*:

```
wget https://ftp.ncbi.nlm.nih.gov/genomes/all/GCF/001/617/565/
GCF_001617565.1_ASM161756v1/GCF_001617565.1_ASM161756v1_genomic.fna.gz
gunzip GCF_001617565.1_ASM161756v1_genomic.fna.gz
```

Parte A:

Tras crear los archivos se trabajó en Rstudio. Se comenzó seteando el working directory para tener los archivos en el ambiente y se cargó la biblioteca *sequinr* para poder leer y analizar las secuencias biológicas. Luego se generaron dos variables con los genomas completos correspondientes a cada cepa utilizando el comando *read.fasta*, el cual permite leer los archivos en formato FASTA.

```
setwd("~/Obligatorio_2024/")
```

```
library(sequinr)
```

```
#Cargo genomas en R
```

```
ecoli_732_genoma <- read.fasta("Ecol_732/Ecol_732.fna")
```

```
ecoli_C11_genoma <- read.fasta("Ecol_C11/Ecol_C11.fna")
```

Luego se calculó el contenido GC de cada replicón en los genomas. Para ello se crearon nuevas variables y se utilizó la función *sapply* para calcular el contenido GC de cada replicón por separado, lo que evita sesgos al unir replicones con diferentes longitudes. Para ello se especificó sobre que lista se debe aplicar la función seguido de la función a aplicar a cada

elemento de la lista, en este caso GC proveniente del paquete de *sequinr*. Tras calcular el contenido GC de cada replicón se hizo un promedio del vector numérico nombrado generado previamente utilizando la función *mean* para cada cepa.

```
#Calculo GC Para E.coli 732
GC_ecoli_732 <- sapply(ecoli_732_genoma, GC)
GC_ecoli_732
```

```
## NZ_CP015138.1 NZ_CP015140.1 NZ_CP015141.1 NZ_CP015142.1 NZ_CP015143.1
##      0.5075065      0.5254193      0.5328498      0.4980354      0.5100065
## NZ_CP015144.1 NZ_CP015139.1
##      0.4614653      0.5212711
```

```
#Promedio
mean(GC_ecoli_732)
```

```
## [1] 0.5080791
```

```
#Calculo GC Para E.coli C11
GC_ecoli_C11 <- sapply(ecoli_C11_genoma, GC)
GC_ecoli_C11
```

```
## NZ_CP010133.1
##      0.5083328
```

```
#promedio
mean(GC_ecoli_C11)
```

```
## [1] 0.5083328
```

Como se puede observar, la cepa Ecol_732 presenta un contenido GC de 50,81% mientras que la cepa C11 tiene un contenido GC de 50,83%, siendo ambos valores muy similares entre sí. Esto resulta coherente, ya que ambas cepas pertenecen a la misma especie (*E. coli*) y por ende tienen una alta conservación en términos de composición genómica, puesto que al ser del mismo linaje tienen una presión evolutiva compartida. Asimismo, el contenido GC afecta la estabilidad genética y la eficiencia de los procesos celulares esenciales, como la replicación y la transcripción, y por ende, en dos cepas que comparten la mayoría de sus funciones biológicas, además de su entorno, estos porcentajes no deberían presentar una gran variación. La leve diferencia entre ambos porcentajes se puede explicar debido a que la cepa Ecol_732, además de su cromosoma principal (el cual de por sí ya cuenta con una pequeña

diferencia de GC con el de C11), tiene 6 plásmidos, lo cual va a aportar al contenido GC total, mientras que, la cepa C11 solo cuenta con el cromosoma principal.

Parte B:

Para poder determinar el número de secuencias codificantes en los genomas de ambas cepas se trabajó con los archivos previamente descargados en Bash correspondientes a los CDSs (archivos con terminación .ffn). Se crearon dos variables con los archivos y se utilizó nuevamente la función *read.fasta* para leerlos.

```
#Carga CDSs en R
ecoli_732_cds <- read.fasta("Ecol_732/Ecol_732_cds.ffn")
ecoli_C11_cds <- read.fasta("Ecol_C11/Ecol_C11_cds.ffn")
```

Posteriormente, como estos archivos solo cuentan con las secuencias codificantes, solo es necesario aplicar la función *length* para saber el número de secuencias en los vectores generados.

```
#Cuento secuencias codificantes para ecoli 732
length(ecoli_732_cds)
```

```
## [1] 5268
```

```
#Cuento secuencias codificantes para ecoli C11
length(ecoli_C11_cds)
```

```
## [1] 5078
```

Por un lado, la cepa Ecol_732 cuenta con 5268 secuencias codificantes, mientras que la cepa C11 posee 5078. Esto quiere decir que la cepa Ecol_732 es capaz de sintetizar más proteínas que la cepa C11, puesto que si bien ambas comparten el mismo genoma central, el genoma accesorio de la cepa Ecol_732 es mayor que el de la cepa C11. De todos modos, cabe recalcar que son números relativamente similares, lo cual tiene coherencia al tratarse de la misma especie.

Parte C:

Para poder comparar el uso de codones sinónimos para la alanina y el triptófano, se decidió trabajar con las variables ya generadas que contienen los archivos de los CDSs, puesto que en las mismas se encuentran todos los codones que posteriormente codificarán todos los aminoácidos, entre ellos alanina y triptófano. Lo primero que se realizó fue utilizar el comando *unlist* para las dos variables previamente mencionadas, con el fin de generar un vector de

caracteres a partir de la lista de los CDSs, ya que la función *uco* (responsable de calcular el uso de codones en una secuencia de ADN o ARN), solo acepta como input secuencias de bases nitrogenadas en formato vector de caracteres. Posteriormente, se aplica la función *uco* para calcular la frecuencia absoluta de cada codón y el resultado se guarda en nuevas variables.

```
#Trabajo con CDSs
S_732 <- unlist(ecoli_732_cds)
S_C11 <- unlist(ecoli_C11_cds)
```

```
#Cuento Los codones
ucoS_732 <- uco(S_732)
ucoS_C11 <- uco(S_C11)
```

Para obtener los nombres de todos los aminoácidos y posteriormente buscar los solicitados en ambas cepas, se utilizó la función *names* para obtener los nombres de cada elemento en el vector *ucoS_732*, el cual contiene todos los codones y su frecuencia absoluta. Cabe aclarar que solo se realiza con uno de los vectores de las cepas, puesto que lo único que se quiere obtener en este paso son los nombres de todos los codones, y no su frecuencia absoluta en esta cepa en específico. Se usa *sapply* con el fin de aplicar a cada elemento del vector (en este caso los nombres de los elementos) la función *s2c*, la cual convierte una cadena de texto (como "gca") en un vector de caracteres (resultando en "g", "c", "a"). Esto se realiza con el fin de darle el input adecuado a la siguiente función a implementar, *translate*, la cual toma como entrada los codones convertidos en vectores de caracteres y los traduce a sus aminoácidos correspondientes. Como la función *translate* da como output los aminoácidos en código de un solo carácter, se implementa la función *aaa* para convertir el código en su nombre semi-completo. Por último, se convirtió los nombres de los aminoácidos obtenidos en un objeto de tipo factor mediante la función *as.factor*, forma más eficiente de almacenar los aminoácidos al tratarse cada aminoácido como una categoría única, por más que la degeneración del código genético causa repeticiones de los mismos. Este vector con los nombres se almacenó en una variable llamada *namesaa*. Luego se buscó en esta variable las posiciones de los nombres de los aminoácidos solicitados ("Ala" y "Trp") mediante la función *grep* y se almacenó su posición en dos variables distintas.

```
#Obtener Los aminoácidos codificados para cada codón
namesaa = as.factor(aaa(translate(sapply(names(ucoS_732),s2c))))
```

```
#Determinar Las posiciones en Las que encontramos Alanina
vectAla <- grep("Ala",namesaa)

#Determinar Las posiciones en Las que encontramos Triptofano
vectTrp <- grep("Trp",namesaa)
```

Como el ejercicio requiere comparar los datos normalizados, se optó por calcular la frecuencia relativa de los codones sinónimos empleando la función *uco* con el índice RSCU (Relative

Synonymous Codon Usage). Esto fue así, ya que el índice RSCU (Relative Synonymous Codon Usage) permite normalizar el uso de los codones sinónimos para cada aminoácido, eliminando el sesgo generado por la abundancia general de un aminoácido en particular. Este índice compara la frecuencia observada de un codón con la frecuencia esperada si todos los codones sinónimos se utilizaran de manera uniforme.

```
#Frecuencia relativa de codones sinonimos
rscuS_732 <- uco(seq=S_732,index="rscu")
rscuS_C11 <- uco(seq=S_C11,index="rscu")
```

Luego, se generaron dos tablas para cada bacteria, una de cada aminoácido. En estas tablas se guardaron únicamente la frecuencia relativa de codones sinónimos de cada codón (gca, gcc, gcg y gct para alanina y tgg para triptofano).

```
#Cuento alaninas en cada bacteria
rscuS_ala_732 <- rscuS_732[vectAla]

rscuS_ala_C11 <- rscuS_C11[vectAla]
```

```
#Cuento triptofano en cada bacteria
rscuS_trp_732 <- rscuS_732[vectTrp]

rscuS_trp_C11 <- rscuS_C11[vectTrp]
```

A partir de estas tablas fue posible generar un gráfico para poder comparar el uso de codones sinónimos para Alanina y Triptófano, pero primero se generó un *data frame* unificado para así poder graficar utilizando *ggplot2*.

```
# Crear un data frame unificado
codones <- c(codones_ala, codones_trp)
especie <- c(especie_ala, especie_trp)
rscuS <- c(rscuS_ala, rscuS_trp)
aminoacido <- c(rep("Alanina", length(codones_ala)), rep("Triptófano", length(codones_trp)))

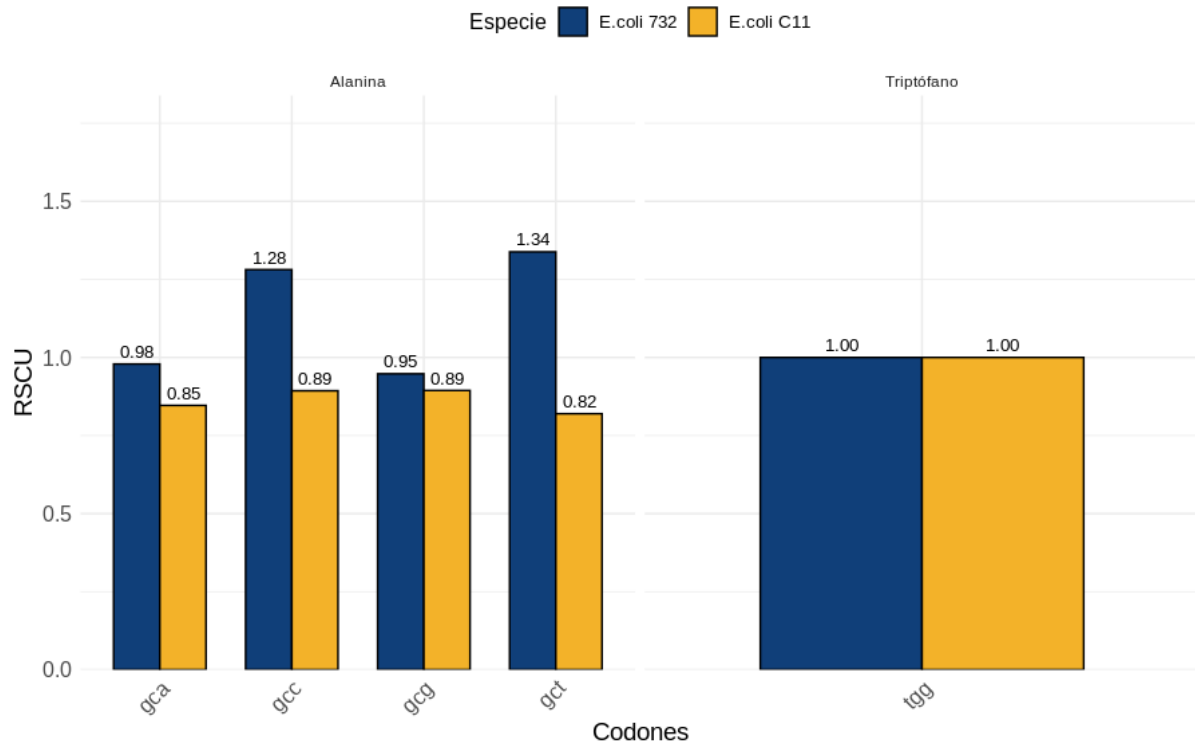
data <- data.frame(codones, especie, rscuS, aminoacido)
```

Finalmente, se graficó el *data frame* unificado de las dos bacterias utilizando *ggplot2*. El comando *geom_text* se usó para añadir las etiquetas con los valores de RSCU sobre las barras, lo que facilita la lectura y comprensión del gráfico. *scale_fill_manual* se utilizó para personalizar los colores de las barras, permitiendo diferenciar claramente las especies

representadas. A través del comando *labs*, se incorporaron tanto un título general para el gráfico como etiquetas descriptivas para los ejes y la leyenda. *facet_wrap* permitió generar paneles separados para cada aminoácido, mientras que el comando *scale_y_continuous* se utilizó para establecer los límites del eje Y, ajustándolos entre 0 y el valor máximo de RSCU más un margen de 0.5. Los demás comandos se emplearon con fines estéticos, como rotar las etiquetas del eje X, centrar el título y ajustar la leyenda, para mejorar la presentación general y la legibilidad del gráfico.

```
# Graficar para frecuencia relativa de codones sinonimos
ggplot(data, aes(fill=especie, y=rscuS, x=codones)) +
  geom_bar(position="dodge", stat="identity", color="black", width=0.7) +
  geom_text(aes(label=sprintf("%.2f", rscuS),
                    position=position_dodge(width=0.7), vjust=-0.5, size=3.5) +
  scale_fill_manual(values=c("#103f79", "#f3b229")) + # BOCA
  labs(title = "Frecuencia relativa de codones sinónimos ",
        x = "Codones",
        y = "RSCU",
        fill = "Especie") +
  facet_wrap(~aminoacido, scales = "free_x") + # Paneles separados por aminoácido
  scale_y_continuous(limits = c(0, max(data$rscuS) + 0.5), expand = c(0,
0)) + # Ajustar rango del eje Y
  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, size = 16, face = "bold"),
    axis.text.x = element_text(angle = 45, hjust = 1, size = 12),
    axis.text.y = element_text(size = 12),
    axis.title.x = element_text(size = 14),
    axis.title.y = element_text(size = 14),
    legend.position = "top",
    legend.title = element_text(size = 12),
    legend.text = element_text(size = 10)
  )
```


Frecuencia relativa de codones sinónimos



A partir de un análisis del gráfico, se puede observar que, para el caso de alanina, se observan diferencias claras entre ambas cepas. En *E. coli* 732, se destaca un uso preferencial del codón GCT (RSCU = 1.34), seguido por GCC (1.28), mientras que GCA y GCG tienen frecuencias más equilibradas (0.98 y 0.95, respectivamente). Por el contrario, en *E. coli* C11, no hay un codón predominante, ya que los valores de RSCU son similares para GCA (0.85), GCC (0.89), GCG (0.89) y GCT (0.82). Esto indica una mayor uniformidad en el uso de codones para alanina en *E. coli* C11. En el caso de triptófano, ambas cepas presentan un RSCU de 1.00, esto es lógico, ya que este aminoácido al estar codificado exclusivamente por el codón TGG no tiene codones sinónimos.

Ejercicio 2:

Parte A:

Una secuencia paróloga es una secuencia que se originó por una duplicación genética dentro de un mismo genoma. Es decir, cuando un gen experimenta una duplicación, el gen original y su copia se consideran parálogos. Estos genes pueden evolucionar de manera independiente después de la duplicación, adquiriendo nuevas funciones o manteniendo funciones similares, pero con variaciones. Teniendo esto en cuenta, para poder saber cuantas secuencias parálogas tiene cada cepa, se decidió realizar un blastp por dos motivos principales. Primeramente, al tener en consideración que para que dos secuencias sean parálogas las proteínas que codifican deben cumplir funciones diferentes, se debe utilizar las secuencias aminoacídicas ya que por más que haya diferencias a nivel de ADN, las mutaciones sinónimas puede que no alteran la proteína final y por ende cumplan la misma función. Otro motivo por el cual se optó por realizar un blastp es porque el mismo es más sensible que un blastn, debido a que blastn compara entre secuencias que solo tienen 4 posibles nucleótidos, lo que limita la diversidad de combinaciones y genera una alta probabilidad de coincidencias falsas o de bajo nivel de identidad, mientras que blastp compara entre secuencias con 20 posibles aminoácidos.

Se trabajó en BASH, y se comenzó creando las bases de datos para ambas cepas. Para ello se utilizó el comando `makeblastdb` y se seleccionaron como input en cada caso los archivos correspondientes para las secuencias aminoacídicas (.faa) de cada cepa, aclarando que la base de datos iba a ser de proteínas.

```
makeblastdb -in Ecol_732_prot.faa -dbtype prot -out ec_732_db
makeblastdb -in Ecol_C11_prot.faa -dbtype prot -out ec_C11_db
```

Tras realizar la base de datos, se procedió a realizar el blastp. Como se estaban buscando secuencias parálogas, el *query* fue la secuencia aminoacídica de la misma cepa que la base de datos, ya que las secuencias parálogas se buscan dentro de una misma cepa. Se estableció que el formato de salida fuera en tabla y que la misma contuviera como columnas, primeramente, el identificador de la secuencia de consulta (*qseqid*) y el identificador de la secuencia encontrada en la base de datos (*sseqid*), los cuales posteriormente fueron utilizados para eliminar los autohits. Se pidió a su vez porcentaje de identidad de la coincidencia (*pident*) y la cobertura de la secuencia de consulta por la secuencia encontrada (*qcovs*) puesto que son los parámetros con los que se solicitaron filtrar los datos. Se utilizó la matriz de sustitución BLOSUM62, matriz estándar para comparar secuencias de proteínas, la cual ofrece un equilibrio adecuado entre sensibilidad y especificidad. A su vez, se utilizó el comando *subject_besthit*, con el fin de retener únicamente la mejor coincidencia para cada secuencia de consulta, eliminando redundancias. Esto asegura que el resultado sea más preciso y reduce el tamaño de los archivos de salida. Por último, se utiliza el comando *eval* $1e-20$, con el fin de setear un umbral estricto para el valor de expectativa que no permita que pasen aquellos hits que tienen altas probabilidades de haberse dado al azar. Cabe mencionar que también se utilizó el comando *num_threads* 2 simplemente para acelerar la búsqueda.

732:

```
blastp -query Ecol_732_prot.faa -db ec_732_db -outfmt '6 qseqid sseqid pident qcovs eval  
ue' -matrix BLOSUM62 -subject_besthit -out paralogos_732 -num_threads 2 -evaluate 1e-20
```

C11:

```
blastp -query Ecol_C11_prot.faa -db ec_C11_db -outfmt '6 qseqid sseqid pident qcovs eva  
lue' -matrix BLOSUM62 -subject_besthit -out paralogos_C11 -num_threads 2 -evaluate 1e-20
```

Tras obtener los dos archivos de salida de los blastp correspondientes a las cepas Ecol_732 y C11, se procedió al filtrado de los resultados en BASH. Primeramente, al utilizarse el mismo archivo para el *query* como para la base de datos, se formaron autohits (las coincidencias donde la secuencia de consulta es la misma que la secuencia encontrada en la base de datos), los cuales hay que eliminar porque un parólogo, por definición, es una copia diferente del gen dentro del mismo genoma. Para poder filtrarlos se utiliza el comando *awk* el cual evalúa si el identificador de consulta (columna 1) es diferente del identificador encontrado (columna 2). En caso de serlo, se redirige la salida filtrada a otro archivo para ser almacenados.

```
awk '$1 != $2 {print}' paralogos_732 > paralogos_732_f  
awk '$1 != $2 {print}' paralogos_C11 > paralogos_C11_f
```

Por último, se realiza un nuevo filtrado por identidad y cobertura para los archivos de las dos cepas. Para ello se vuelve a utilizar *awk* y se le pide que imprima solo aquellas líneas cuya columna 3 (porcentaje de identidad) sea mayor al 80% y a su vez que se cumpla la condición de que la columna 4 (cobertura del query) sea mayor al 90%. Todos los hits que cumplen con ambas condiciones son redirigidos a un nuevo archivo que contiene exclusivamente aquellas coincidencias que tienen una similitud alta y una cobertura significativa. Finalmente, se cuentan las coincidencias en cada archivo filtrado utilizando el comando *wc -l*, que cuenta el número de líneas en un archivo, las cuales en este caso corresponden al número de hits.

```
awk '$3>80 && $4>90 {print}' paralogos_732_f > paralogos_732_f2  
wc -l paralogos_732_f2  
#Esto da: wc -l paralogos_732_f2 = 190  
  
awk '$3>80 && $4>90 {print}' paralogos_C11_f > paralogos_C11_f2  
wc -l paralogos_C11_f2  
#Esto da: wc -l paralogos_C11_f2 = 136
```

Para el caso de la cepa Ecol_732, la misma cuenta con 190 pares de secuencias parálogas que cumplen con los criterios establecidos, mientras que la cepa C11 cuenta con 136.

Parte B:

Se comenzó descargando la secuencia del gen *atpA*, que codifica la subunidad A de la ATP sintasa, de la base de datos UniProt mediante el siguiente comando.

```
wget https://rest.uniprot.org/uniprotkb/P0ABB0.fasta
```

Se seleccionó la herramienta blastp para este análisis porque la secuencia descargada de UniProt corresponde a una secuencia de aminoácidos. Esto la hace más adecuada para su comparación con otras secuencias proteicas, ya que blastp permite identificar similitudes a nivel de aminoácidos directamente, lo que es crucial para capturar homología funcional. Además, las proteínas suelen ser más conservadas evolutivamente que las secuencias de ADN, lo que facilita detectar relaciones entre genes que puedan tener diferencias a nivel de nucleótidos, pero la misma función. Asimismo, como se mencionó previamente, blastp es una herramienta más sensible, puesto que por la naturaleza de los aminoácidos en contraposición con los nucleótidos, existe una menor probabilidad de encontrar coincidencias aleatorias entre secuencias. Adicionalmente, blastp también captura mejor las divergencias evolutivas que no afectan la funcionalidad proteica, como las mutaciones sinónimas.

Para realizar los blastp se utilizaron los mismos comandos descritos en la parte A, con la diferencia que en los blastp de ambas cepas el *query* fue el archivo FASTA que contiene la secuencia de atpA. Se trabajó con las bases de datos previamente creadas, puesto que ambas fueron creadas con los archivos correspondientes a las secuencias aminoacídicas de las dos cepas correspondientes.

```
blastp -query P0ABB0.fasta -db ec_732_db -outfmt '6 qseqid sseqid pident qcovs evaluate'
-matrix BLOSUM62 -subject_besthit-evalue 1e-20 -num_threads 16 -out atpA_732

blastp -query P0ABB0.fasta -db ec_C11_db -outfmt '6 qseqid sseqid pident qcovs evaluate'
-matrix BLOSUM62 -subject_besthit-evalue 1e-20 -num_threads 16 -out atpA_C11
```

Tras obtener los archivos resultantes del blastp se procedió al filtrado de los mismos. Se aplicaron los filtros requeridos utilizando el comando *awk* para seleccionar las coincidencias con un porcentaje de identidad (columna 3) mayor o igual al 50% y con una cobertura del query (columna 4) mayor o igual al 90%. Los resultados de los filtrados se redirigieron a nuevos archivos y se utilizó el comando *wc -l* para contar el número de líneas en los archivos filtrados, representando el número de coincidencias aceptadas.

```
awk '$3>80 && $4>90 {print}' paralogos_732_f > paralogos_732_f2
wc -l paralogos_732_f2
#Esto da: wc -l paralogos_732_f2 = 1

awk '$3>80 && $4>90 {print}' paralogos_C11_f > paralogos_C11_f2
wc -l paralogos_C11_f2
#Esto da: wc -l paralogos_C11_f2 = 1
```

Para ambas cepas existe una única secuencia de copias de atpA que cumplen con las condiciones de filtrado establecidas.

Ejercicio 3:

Parte D:

La utilidad de enraizar un árbol filogenético radica en que el hacerlo permite identificar el ancestro común más reciente de todos los taxones representados en el árbol, añadiendo un contexto evolutivo y direccionalidad a la evolución. Esto permite interpretar las relaciones evolutivas en términos de descendencia y orden temporal de las divergencias. La raíz identifica cuál fue el punto de partida evolutivo, determinando así qué características son ancestrales y cuáles son derivadas entre los diferentes taxones. Mientras tanto, un árbol sin raíz simplemente muestra las relaciones de similitud entre los taxones sin indicar en qué dirección ocurrió la evolución. Asimismo, un árbol enraizado permite una mayor distinción de los clados monofiléticos y diferenciar grupos hermanos, puesto que un árbol sin raíz los grupos relacionados se presentan sin un orden claro.



En este ejemplo generado en R, se observa que en el árbol sin raíz no hay una dirección de la evolución. En contraposición, el árbol enraizado permite identificar un orden temporal en los eventos evolutivos. En este caso, se puede observar que primero hubo un cambio evolutivo que separó al organismo t4 del resto de la filogenia, seguido por otro cambio evolutivo que llevó a t2 a divergir de t1 y t3, y finalmente una divergencia evolutiva entre t1 y t3. Además, el árbol enraizado permite identificar a t4 como el grupo externo (outgroup), lo que no es posible con el árbol sin raíz. Esto es fundamental para determinar qué caracteres son ancestrales y cuáles son derivados, estableciendo una dirección evolutiva. El árbol enraizado también facilita la identificación de grupos monofiléticos, parafiléticos y polifiléticos, proporcionando una representación más clara de las relaciones filogenéticas y los eventos de especiación. Por otro lado, el árbol sin raíz solo muestra relaciones relativas entre los organismos. Por ejemplo, deja claro que t1 y t3 están más relacionados entre sí que con el resto, y que hay menos cambios evolutivos entre t4 y t2 que entre t4 y t1 o t3. Sin embargo, no aporta información sobre ancestros comunes o el orden temporal de los eventos evolutivos.

Parte E:

Un alto valor de bootstrap en un nodo de un árbol filogenético indica que hay una alta confiabilidad estadística en la relación evolutiva que representa ese nodo. El bootstrap es una técnica estadística que consiste en realizar múltiples repeticiones de los datos originales, creando nuevos conjuntos de datos mediante el muestreo con reemplazo. A partir de estas muestras, se generan varios árboles filogenéticos, y se observa con qué frecuencia aparece cada nodo en estos árboles. Si un nodo se repite en un gran porcentaje de los árboles generados (por ejemplo, un 90% o más), esto sugiere que la relación evolutiva en ese punto es robusta y está bien respaldada por los datos.

La importancia de este valor en filogenia radica en que un alto valor de bootstrap refleja una mayor confianza en la hipótesis de que los taxones representados por ese nodo están relacionados de la manera en que se muestra en el árbol. En cambio, un valor bajo de bootstrap indica que la relación es más incierta, lo que podría llevar a reinterpretaciones o revisiones si se obtienen nuevos datos o se emplean otros métodos. Así, el valor de bootstrap ayuda a fortalecer las conclusiones evolutivas al proporcionar una medida de la consistencia de las relaciones propuestas entre los organismos estudiados y así validar la estabilidad de las agrupaciones propuestas por un árbol filogenético.

Parte F:

El gráfico muestra el resultado de un análisis de componentes principales (PCA), que permite visualizar la variabilidad genética entre dos linajes de una especie, representados por colores. La separación de los puntos en dos grupos bien definidos indica que existen diferencias genómicas claras entre ambos linajes, basadas en la presencia (1) o ausencia (0) de un grupo de genes en cada genoma. La Componente Principal 1 (PC1) es la que explica la mayor parte de la variabilidad genética entre los linajes, ya que los grupos se separan principalmente a lo largo de este eje. Esto sugiere que las diferencias más relevantes en la composición genética de los linajes están capturadas por PC1. Por otro lado, la Componente Principal 2 (PC2) muestra menor variabilidad entre los grupos, indicando que contribuye en menor medida a la separación entre los linajes, aunque podría reflejar diferencias adicionales dentro de cada grupo. Asimismo, se podría decir que el linaje rojo es más similar entre sí que el linaje azul. Esto se observa en el gráfico porque los puntos del linaje rojo están más agrupados y ocupan un espacio menor en ambas componentes principales (PC1 y PC2). Esto indica una menor variabilidad genética dentro del linaje rojo, lo que sugiere que las muestras de este grupo son más homogéneas. Por el contrario, los puntos del linaje azul están más dispersos, lo que refleja una mayor variabilidad genética dentro de este grupo.